

Chess API

Project Architecture & Design Reference

ASP.NET Core 8

React + TypeScript

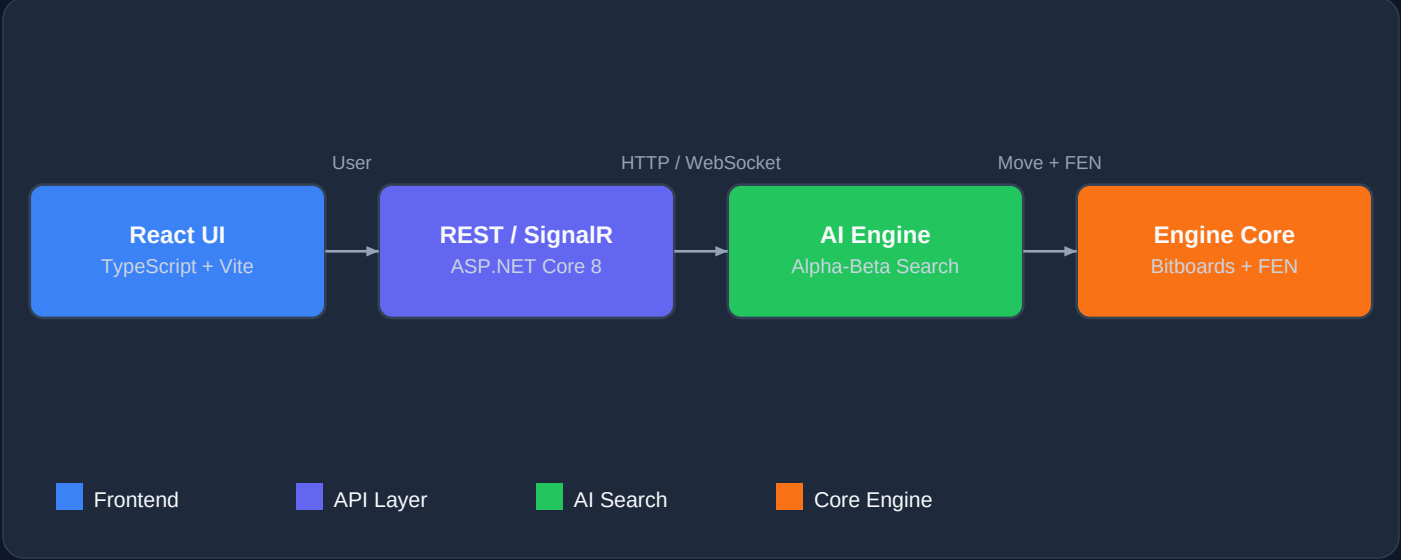
Bitboard Engine

SignalR

Project Overview

Chess API is a full-stack chess application with a custom bitboard chess engine, an ASP.NET Core REST/SignalR backend, and a React + TypeScript frontend. Players compete against an AI that uses iterative-deepening Alpha-Beta search with advanced pruning and real-time analysis streaming.

System Architecture



Tech Stack

Layer	Technology	Purpose
Frontend	React 19 + TypeScript + Vite	Interactive chess board & analysis UI
API	ASP.NET Core 8 + SignalR	REST endpoints & real-time move streaming
AI Engine	C# Alpha-Beta Search	Move search with iterative deepening
Core Engine	C# Bitboards	Board state, move generation, Zobrist hashing
Testing	xUnit 2.9	Board, perft, evaluation & search unit tests

API Reference

REST Endpoints — Game Management

Method	Route	Description
POST	/api/game/start	Start a new game; choose white or black
POST	/api/game/move	Submit a move (UCI notation); receive AI response
GET	/api/game/{id}/state	Retrieve current board state & move history
POST	/api/game/{id}/resign	Resign the active game

REST Endpoints — Engine Analysis

Method	Route	Description
POST	/api/engine/move	Find best move for a FEN (configurable depth / time)
POST	/api/engine/evaluate	Return static evaluation score (centipawns)
GET	/api/engine/perft/{depth}	Perft node-count test at given depth

SignalR Hub — /hubs/engine

Direction	Event / Method	Payload
Client → Server	FindBestMove(fen, depth, timeMs)	Triggers async search
Server → Client	searchProgress	{ depth, score, nodes, timeMs, pv }
Server → Client	bestMove	{ move (UCI), score (cp) }
Server → Client	searchError	{ message }

Engine Design

Move Encoding



Each chess move is packed into a single 32-bit integer. This eliminates heap allocations during move generation and keeps the move list cache-friendly.

Search Algorithm



Search Parameters

Parameter	Value	Notes
Max depth	30 plies	Iterative deepening starts at depth 1
Time limit	60 seconds	Per move; deepening stops when limit reached
Aspiration window	± 50 cp	Widened on fail-high / fail-low
LMR trigger	$\text{depth} \geq 3, \text{move} > 4$	Reduces non-capture quiet moves
TT size	8,388,608 entries	Depth-preferred replacement
Killer moves	2 per ply	Non-capture moves that caused β cut-offs

Evaluation & Data Flow

Position Evaluation

Term	Detail	Impact
Material	P=100 N=320 B=330 R=500 Q=900 K=20 000	Primary signal
Piece-square tables	Separate middlegame & endgame PSTs per piece	Positional bonuses
Pawn structure	Isolated / doubled penalties; passed pawn bonus (scales by rank)	Endgame-weighted
King safety	Count enemy attacks near king; endgame → centralize king	Critical in MG
Tapered eval	Score = (MG × phase + EG × (24−phase)) / 24	Smooth MG→EG blend

Game Data Flow

Step	Actor	Action	Output
1	Player	Select color, click Start	POST /api/game/start
2	API	Init board, store game state	gameId + starting FEN
3	Player	Drag piece on board	POST /api/game/move (UCI)
4	API	Validate move, apply to board	Player FEN
5	AI Search	Iterative deepening α-β	Best move (UCI)
6	API	Apply AI move, check game status	AI FEN + status
7	SignalR	Stream search progress events	depth / score / PV
8	Frontend	Update board, eval bar, PV panel	Visual update

Key Design Decisions

In-Memory Storage

- ConcurrentDictionary<string, GameState>
 - No database — state lost on restart
 - TranspositionTable is DI singleton
 - Search is DI transient (one per request)
- Scale-out would require a distributed cache

Frontend Architecture

- chess.js validates moves client-side
 - react-chessboard renders the board
 - SignalR streams analysis in real time
 - Two-screen flow: setup → game
- Analysis panel updates per depth level

Project Structure

Repository Layout

Path	Purpose
Chess.Engine.Core/	Bitboard engine — board, moves, Zobrist, move gen
Chess.Engine.AI/	Alpha-Beta search, evaluation, transposition table
Chess.API/	ASP.NET Core app — controllers, SignalR hub, DTOs
chess-ui/	React + TypeScript frontend (Vite)
Chess.Tests/	xUnit tests — board, perft, evaluation, search, TT
pseudocode/	Human-readable design documents for each layer
README.md	Project overview, setup guide, API reference

Key Source Files

File	Layer	Responsibility
Board.cs	Core	Board state, FEN parsing, MakeMove / UndoMove
Bitboards.cs	Core	Magic bitboard attack tables (rook, bishop, queen, knight, king, pawn)
MoveGenerator.cs	Core	Pseudo-legal generation → legal filter via IsInCheck
Zobrist.cs	Core	Incremental Zobrist hashing for position identity
Search.cs	AI	Iterative-deepening α - β with LMR, killers, history, aspiration
Evaluation.cs	AI	Tapered material + PST + pawn structure + king safety
TranspositionTable.cs	AI	8.4 M entry hash table with depth-preferred replacement
GameController.cs	API	POST start / move / resign; in-memory game store
EngineController.cs	API	POST best-move / evaluate; GET perft
EngineHub.cs	API	SignalR hub streaming search progress to browser
App.tsx	UI	Root component — game state, move handling, SignalR wiring
engineHub.ts	UI	SignalR client wrapper (connect, analyze, events)
gameApi.ts	UI	REST client helpers (startGame, makeMove, resignGame)